

Problem Set 10: Solutions

SPEA V706 Math Camp | Module 10 · Friday, August 14

Part A. Concepts

A1. The sampling distribution of a mean

- a. $E[\bar{Y}_n] = \mu = 20$. This is **unbiasedness**: across repeated samples the estimator centers on the estimand.
- b. $\text{Var}(\bar{Y}_n) = \sigma^2/n = 36/36 = 1$.
- c. $\text{se}(\bar{Y}_n) = \sigma/\sqrt{n} = 6/6 = 1$.
- d. Set $6/\sqrt{n} = 0.5$, so $\sqrt{n} = 12$ and $n = 144$. Cutting the standard error in half took four times the data.
- e. We cannot say. Nothing here tells us the shape of the population, only its first two moments. Two cases:
- If the population is Normal, then $\bar{Y}_n$ is exactly $N(20, 1)$ at any n , because a linear combination of Normals is Normal.
 - If it is not, then $\bar{Y}_n$ is only **approximately** Normal, by the CLT, and $n = 36$ may or may not be enough depending on how ugly the population is.

Either way the mean and the standard error in (a) through (c) are correct. Those come from E1–E3 and V2/V5, and they need no distributional assumption beyond iid draws with finite variance.

A2. The rate

- a. Let n_1 give $\text{se}_1 = \sigma/\sqrt{n_1}$ and ask for n_2 with $\text{se}_2 = \text{se}_1/2$:

$$\frac{\sigma}{\sqrt{n_2}} = \frac{1}{2} \cdot \frac{\sigma}{\sqrt{n_1}} \implies \sqrt{n_2} = 2\sqrt{n_1} \implies n_2 = 4n_1$$

- b. They want the standard error cut by a factor of 4, so n must grow by a factor of $4^2 = 16$. They need $16 \times 1,000 = 16,000$ observations, which is **15,000 more** than they have.
- c. Precision is expensive and gets more expensive as you go. The first thousand observations buy far more than the sixteenth thousand does. When a design is underpowered, adding a bit more data rarely rescues it, and it is usually better to find a setting with more variation in the treatment or a better-measured outcome than to grind out a marginal increase in n .

A3. True or false

a. False. This is the gambler's fallacy. The coin has no memory, and each flip stays 50/50. The LLN works by *swamping* the early run with an enormous number of later flips, not by correcting it. The running proportion converges to 0.5 while the running *difference* between heads and tails can keep growing.

b. False. The CLT is about the distribution of the **sample mean**, not about Y . Wages do not become Normal because you collected a lot of them.

c. False. That is the point of the theorem. Any population with finite variance will do, and the M5 deck demonstrates it on a badly skewed Exponential.

d. False. Unbiasedness is a property of the estimator across *repeated samples*, not a promise about the one sample you have. An unbiased estimator with a huge variance can be wildly off in any given draw. Bias and variance are separate questions.

e. True. $\text{Var}(\bar{Y}_n) = \sigma^2/n \rightarrow 0$ as $n \rightarrow \infty$, which is the engine of the LLN.

f. False. They describe different objects. The standard deviation of Y describes how spread out the **data** are, and it does not shrink with n ; collecting more workers does not make wages less variable. The standard error describes how spread out an **estimate** is across hypothetical repeated samples, and it does shrink, at rate $1/\sqrt{n}$.

A4. Unbiasedness, line by line

$$\begin{aligned} E[\bar{Y}_n] &= E\left[\frac{1}{n} \sum_{i=1}^n Y_i\right] \\ &= \frac{1}{n} E\left[\sum_{i=1}^n Y_i\right] \quad (\text{E2: the constant } 1/n \text{ slides out}) \\ &= \frac{1}{n} \sum_{i=1}^n E[Y_i] \quad (\text{E3: the expectation of a sum splits}) \\ &= \frac{1}{n} \sum_{i=1}^n \mu \quad (\text{identically distributed: every } E[Y_i] = \mu) \\ &= \frac{1}{n} n \mu = \mu \end{aligned}$$

Which part of iid did we use? Only the *identically distributed* part, at the fourth line, to say every draw has the same mean μ . We never used **independence**. E3 splits the expectation of a sum whether or not the terms are related.

That is worth remembering: the sample mean stays unbiased under dependence, say with clustered or serially correlated data. What breaks is the **variance**. The derivation of σ^2/n used V5 and needed every covariance term to vanish, which is where independence did the work. Dependent data leaves the estimate centered correctly and makes the usual standard error wrong, often badly too small.

A5. When the approximation is poor

a. Two features:

- **Heavy skew.** A distribution with a long right tail, like income or firm size, needs a much larger n before the sample mean looks symmetric. The Exponential in the deck still shows visible skew at $n = 5$.
- **Heavy tails, or a rare binary outcome.** With very fat tails the variance may be enormous or undefined, and the CLT is slow or fails outright. For a 0/1 variable with $p = 0.002$, a sample of 30 almost always contains zero successes, and no Normal approximation is going to save you. The rough guide people use is np and $n(1 - p)$ both above 10.

b. **Dependence between observations.** Clustered sampling, repeated measurements on the same person, spatial correlation, or time-series autocorrelation all break the independence that made the cross terms in V5 vanish. The formula σ^2/n then understates the true sampling variance, sometimes by a lot, because you have less independent information than your row count suggests. Clustered standard errors exist to repair exactly this.

c. $se = 3.69/\sqrt{526} \approx 0.161$.

The \$3.69 says that individual workers' wages are spread out by about \$3.69 around the sample average. The \$0.161 says that the sample *average itself* would move by only about 16 cents from one sample of 526 workers to another. The second is more than twenty times smaller, because averaging 526 workers cancels most of the individual noise.

Part B. R

B1. Build a project from scratch

```
sampling_lab/  
  sampling_lab.Rproj  
  main.r  
  .gitignore  
  code/  
    01_simulate.R  
    02_analyze.R  
  outputs/  
    figures/  
  raw_data/
```

A .gitignore that does the job:

```
.Rproj.user/  
.Rhistory  
.RData  
raw_data/
```

e. `here()` resolves paths from the project root, wherever that project happens to sit, so the same code runs on your laptop, on a coauthor's machine, and on a cluster. A hard-coded `setwd()` names a folder that exists on exactly one computer, and it breaks the moment you rename a directory, share the project, or open it from a different working directory. A replication package containing `setwd("/Users/coady/Desktop/...")` is a replication package that does not replicate.

B2. code/01_simulate.R

```
pacman::p_load(data.table, ggplot2, here)
set.seed(20260814)

n_sims <- 5000
sizes <- c(5, 30, 200)

# For each n: fill an n_sims x n matrix with draws, then average across each
# row. One vectorized call per size beats 5000 trips through replicate().
sims <- rbindlist(lapply(sizes, function(n) {
  draws <- matrix(rexp(n_sims * n, rate = 1), nrow = n_sims, ncol = n)
  data.table(n = n, xbar = rowMeans(draws))
}))

nrow(sims)
```

```
[1] 15000
```

```
head(sims, 3)
```

	n	xbar
	<num>	<num>
1:	5	2.0438261
2:	5	0.7773323
3:	5	1.8355719

Then hand the result to the next script through a file rather than through the workspace:

```
saveRDS(sims, here("outputs", "sampling_sims.rds"))
message("01_simulate.R complete")
```

b. 15,000 rows, 5,000 for each of the three sample sizes, with columns `n` and `xbar`.

B3. code/02_analyze.R

```
summary_tab <- sims[, .(
  mean_xbar = mean(xbar),
  sd_xbar   = sd(xbar),
  theory_se = 1 / sqrt(n[1])
), by = n]
```

```
summary_tab
```

	n	mean_xbar	sd_xbar	theory_se
	<num>	<num>	<num>	<num>
1:	5	0.9994979	0.45006239	0.44721360
2:	30	0.9978176	0.18110604	0.18257419
3:	200	0.9995627	0.07177183	0.07071068

a. The three means are 0.9995, 0.9978, and 0.9996, all essentially 1. The LLN predicts that the sample mean converges to the population mean, which here is 1 for an Exponential with rate 1. Note that the mean is close to the truth at *every* n , including $n = 5$, because the estimator is unbiased. What changes with n is the spread, not the center.

b. Simulated against theoretical $1/\sqrt{n}$:

n	sd of $\bar{Y}$	$1/\sqrt{n}$
5	0.450	0.447
30	0.181	0.183
200	0.072	0.071

The match is close at all three sizes, which is a check on the algebra from this morning rather than on the CLT.

c. The sample sizes differ by a factor of $200/5 = 40$, so the standard errors should differ by $\sqrt{40} \approx 6.32$:

```
summary_tab[n == 5, sd_xbar] / summary_tab[n == 200, sd_xbar]
```

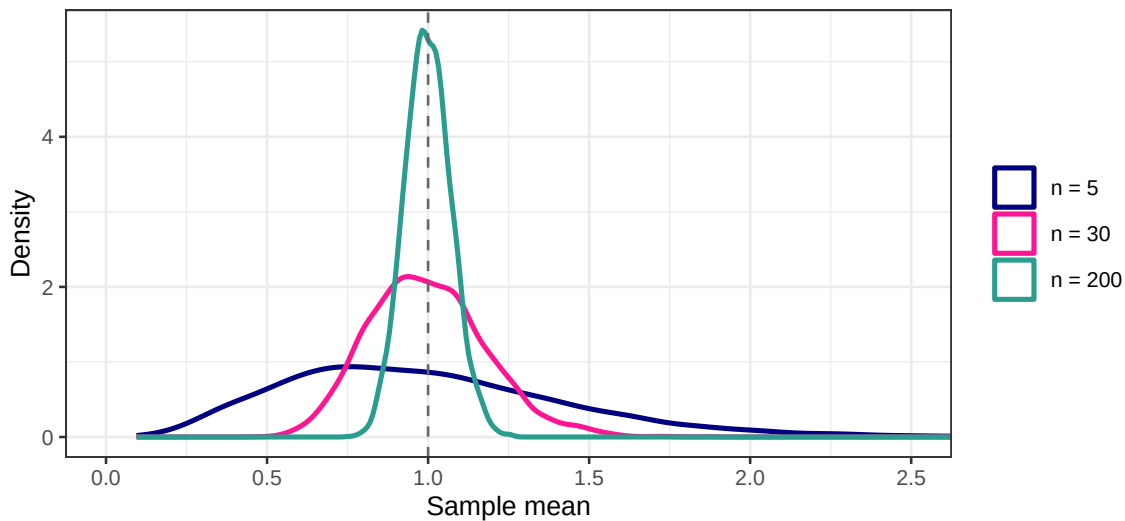
```
[1] 6.270739
```

The simulation gives about 6.27, against a predicted 6.32. The small gap is Monte Carlo noise; raise `n_sims` and it shrinks. Forty times the data bought only a sixfold improvement in precision, which is the $1/\sqrt{n}$ rate being expensive.

d. The figure:

```
sims[, n_lab := factor(n, levels = sizes, labels = paste0("n = ", sizes))]

p <- ggplot(sims, aes(xbar, colour = n_lab)) +
  geom_density(linewidth = 0.9) +
  geom_vline(xintercept = 1, linetype = "dashed", colour = "grey40") +
  scale_colour_manual(values = c("#000080", "#FF1493", "#2A9D8F")) +
  coord_cartesian(xlim = c(0, 2.5)) +
  labs(x = "Sample mean", y = "Density", colour = NULL) +
  theme_bw(base_size = 10)
p
```



```
ggsave(here("outputs", "figures", "sampling_dists.png"),
  p, width = 8, height = 4.5, dpi = 300)
```

B4. Tie it together

```
# main.r
pacman::p_load(here)

message("== 01_simulate")
source(here("code", "01_simulate.R"))

message("== 02_analyze")
source(here("code", "02_analyze.R"))

message("== pipeline complete")
```

b, c. The test in (c) is the real one. Deleting the intermediate `.rds` and the figure and rebuilding from `main.r` proves that every output traces back to code, with nothing depending on an object

you happened to create in the console three hours ago. If it fails, the missing line is almost always a `recode` or a `filter` you typed interactively. Do this before you send results to anyone.

Two habits worth forming now: `set.seed()` at the top of any script that draws random numbers, so the simulation reproduces exactly, and `message()` rather than `print()` for progress, so status text goes to `stderr` and stays out of your captured output.

B5. Read your own figure

- a. At $n = 5$ the sampling distribution is still clearly right-skewed. It has inherited the shape of the Exponential population, with a long tail toward large values.
- b. By $n = 30$ it already looks close to symmetric, and at $n = 200$ it is a tight, clean bell.
- c. Each $\bar{Y}$ is an average of n draws, and averaging cancels the extremes: an unusually large draw has to be offset by the others, and its influence is only $1/n$ of the total. The CLT says that this cancellation drives the standardized average toward $N(0, 1)$ whatever the shape of the population, so long as it has finite variance. It would fail if the population had **infinite or undefined variance**, as with a Cauchy distribution, where the sample mean never settles down and is no better an estimator than a single draw. It would also fail in practice if the draws were **not independent**, since then the averaging cancels much less than you think.

Check yourself.

1. A **standard deviation** describes the spread of a variable in the population or the data, and it does not shrink as you collect more observations. A **standard error** describes the spread of an *estimator* across repeated samples, and it shrinks at rate $1/\sqrt{n}$. The standard error of the mean happens to equal $\sigma/\sqrt{n}$, which is why the two get confused.
2. Because averaging cancels noise. Pull the $1/n$ out of the variance and it comes out squared as $1/n^2$ (V2), while the variance of the sum of n independent draws is only $n\sigma^2$ (V5, with every cross term equal to zero). The result is $n\sigma^2/n^2 = \sigma^2/n$.
3. The LLN says the estimator lands on the truth, and says nothing about how far off it is at any finite n . The CLT supplies the **shape and scale** of the error, $\bar{Y}_n \approx N(\mu, \sigma^2/n)$, which is what lets you build a confidence interval, compute a p -value, or state how precise an estimate is. Without it you would know your estimator works eventually and could say nothing about the answer you actually have.